

UniFecaf - Análise de Desenvolvimento de Sistemas

Pipeline de Dados com IoT e Docker

Luiz Filipe Santiago Pereira

Disruptive Architectures - IoT, Big Data e IA

Documentação teórica do Projeto

1. Contextualização do Projeto

O presente projeto tem como objetivo desenvolver um pipeline completo de dados para processar leituras de temperatura provenientes de dispositivos IoT.

Dispositivos IoT geram volumes contínuos de dados de sensores de temperatura que precisam ser coletados, armazenados e analisados. Sem infraestrutura adequada, esses dados se perdem ou ficam inacessíveis para cada tomada de decisão.

Em cenários reais, sensores de IoT geram grande volume de dados que precisam ser coletados, armazenados e analisados de forma eficiente. Este trabalho simula cenário utilizando um dataset real do Kaggle ("Temperature Readings: IoT Devices"), processando os dados com Python, armazenando em banco de dados PostgreSQL (via Docker) e apresentando os resultados em um dashboard interativo desenvolvido com Streamlit.

2. Objetivos

- Configurar ambiente com Docker + PostgreSQL
- Ingerir dados do arquivo CSV para o banco de dados
- Criar 3 Views SQL para análise
- Desenvolver dashboard interativo com Streamlit
- Extrair insights úteis a partir dos dados

3. Tecnologias Utilizadas

Tecnologia	Função
Python	Linguagem principal do pipeline
PostgreSQL	Banco de dados relacional
Docker	Containerização do banco de dados
SQLAlchemy	Conexão do Python com o PostgreSQL
Streamlit	Criação do Dashboard

pandas	Leitura e manipulação de dados
Plotly	Gráficos interativos
DBeaver	Administração do banco
Git/GitHub	Versionamento e entrega do código

4. Passos Realizados

Passo 1 - Criar repositório no GitHub

Criado repositório público com o nome {luiz1999/IoTBigData} no GitHub, inicializando com README.md.

Passo 2 - Configurar ambiente

Banco de dados PostgreSQL provisionado via Docker:

```
docker run --name postgres-iot -e POSTGRES_PASSWORD=senha -p 5432:5432 -d postgres
```

Conexão configurada no arquivo .env na raiz do projeto:

```
DATABASE_URL=postgresql://postgres:senha@localhost:5432/postgres
```

Dependências instaladas:

```
pip install streamlit pandas sqlalchemy psycopg2-binary plotly python-dotenv
```

Passo 3 - Ingestão dos dados

O arquivo IOT-temp.csv foi importado para o PostgreSQL via DBeaver (Import Data na tabela temperature_readings). Totalde 97.606 registro inseridos.

Passo 4 - Criação das views SQL

Três views foram criadas no DBeaver para consultas otimizadas pelo dashboard.

Passo 5 - Dashboard Streamlit

Arquivo main.py desenvolvido com 3 gráficos interativos carregados diretamente das views SQL. Executado com:

```
streamlit run src/main.py
```

Passo 6 - Git Push

```
git add . | git commit -m 'feat: pipeline IoT' | git push
```

Passo 7 - Deploy no Render.com

Web Service criado no Render.com conectado ao repositório GitHub.
Comando de inicialização configurado:

```
streamlit run main.py --server.port $PORT --server.address 0.0.0.0
```

5. Views SQL Criadas

View 1: avg_temp_por_dispositivo

```
CREATE OR REPLACE VIEW avg_temp_por_dispositivo AS
SELECT
    "room_id/id" AS device_id,
    ROUND(AVG(temp)::numeric, 2) AS avg_temp,
    COUNT(*) AS total_leituras
FROM temperature_readings
GROUP BY "room_id/id"
ORDER BY avg_temp DESC;
```

Conta o total de leituras por local (In/Out). Usada no grafico de pizza para mostrar a proporcao de leituras internas vs externas. Resultado: maior volume de leituras externas.

View 2: leituras_por_hora

```
CREATE OR REPLACE VIEW leituras_por_hora AS
SELECT
    EXTRACT(HOUR FROM TO_TIMESTAMP(noted_date, 'DD-MM-YYYY
HH24:MI'))::int AS hora,
    COUNT(*) AS total_leituras
FROM temperature_readings
```

```
GROUP BY hora  
ORDER BY hora;
```

Conta leituras por hora do dia (0 a 23). Permite identificar padroes de atividade dos sensores. O filtro ≥ 0 remove registros com hora invalida. Resultado: pico de atividade as 14h com mais de 7.000 leituras.

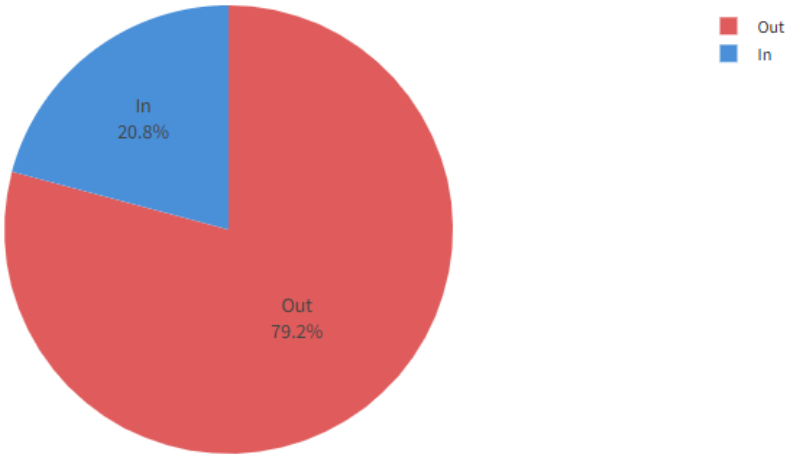
View 3: temp_max_min_por_dia

```
CREATE OR REPLACE VIEW temp_max_min_por_dia AS  
SELECT  
    DATE(TO_TIMESTAMP(noted_date, 'DD-MM-YYYY HH24:MI')) AS  
data,  
    ROUND(MAX(temp)::numeric, 2) AS temp_max,  
    ROUND(MIN(temp)::numeric, 2) AS temp_min,  
    ROUND(AVG(temp)::numeric, 2) AS temp_media  
FROM temperature_readings  
GROUP BY data  
ORDER BY data;
```

Mostra a temperatura maxima e minima registrada por dia. Permite identificar variacao climatica ao longo do periodo. Resultado: maior amplitude termica em setembro/outubro de 2018, com maximas chegando a 51 graus C

6. Visualização do Dashboard

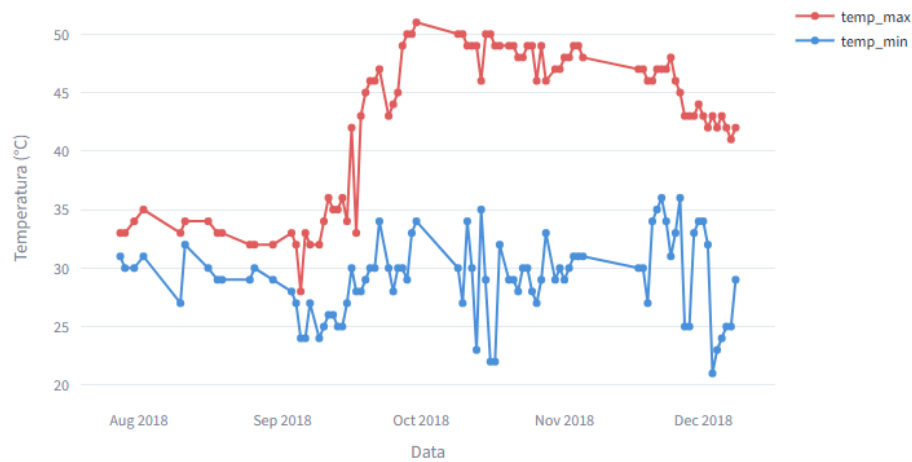
1. Distribuição de Leituras: Interno vs Externo



2. Leituras por Hora do Dia



3. Temperaturas Máximas e Mínimas por Dia



Projeto Disruptive Architectures: IoT, Big Data e IA | UniFECAF

7. Principais Insights

Insights obtidos dos dados

- O dispositivo "Room Admin" registrou temperatura média de aproximadamente **35°C**.
- Existe alta variação térmica ao longo do dia (picos entre 15h e 17h).
- As temperaturas mínimas ficam frequentemente abaixo de 25°C e máximas acima de 50°C em alguns dias.
- O volume de leituras não é uniforme ao longo do dia, indicando possível padrão de atividade dos sensores.

Aplicações Práticas

- Manufatura: monitoramento contínuo de equipamentos com alertas automáticos quando temperatura ultrapassa limites.
- Saúde: controle de câmaras frias e salas cirúrgicas com relatórios automáticos de conformidade.
- Agronegócio: monitoramento de estufas e silos para otimizar irrigação e armazenamento.
- Data centers: identificação de hot spots em racks de servidores para acionamento de sistemas de resfriamento.

8. Conclusão

O projeto permitiu a aplicação prática dos conceitos de IoT, Big Data e integração de ferramentas modernas, demonstrando a capacidade de construir um pipeline completo desde a ingestão até a visualização dos dados.